Istanbul Technical University
Department of Computer Engineering

BLG481E – AI Accelerators
Project 1: Unsupervised Learning with Convolutional AutoEncoders

Team Leader: Ali Eren Çiftçi - 150220022
Team Member: Yavuz Selim Kara - 150220032

March 12, 2026

Contents

1 Introduction **2**

2 Dataset **2**

3 Autoencoder Architecture **2**
3.1 Encoder . . . 3
3.2 Decoder . . . 3
3.3 Architecture Summary . . . 4

4 Training **4**
4.1 Training Setup . . . 4
4.2 Train vs. Test Loss Monitoring . . . 5
4.3 Loss Curves . . . 5

5 Reconstruction Results **6**
5.1 Qualitative Analysis . . . 6

6 Latent Space Visualisation **8**
6.1 PCA Visualisation . . . 8
6.2 t-SNE Visualisation . . . 9

7 Clustering with K-Means **10**
7.1 Clustering Procedure . . . 10
7.2 Evaluation Metrics . . . 10
7.3 Metric Results . . . 11
7.4 Confusion Matrices . . . 12

8 Discussion **13**
8.1 Reconstruction vs. Clustering Trade-off . . . 13
8.2 Challenges . . . 13

9 Conclusion **14**

A Implementation Details **14**

1 Introduction

In this project, we implement a Convolutional AutoEncoder (CAE) on the MNIST handwritten digit dataset to perform unsupervised representation learning. AutoEncoders are neural networks trained to reconstruct their input through a low-dimensional *latent space*, effectively learning a compressed representation without any label supervision.

Our primary objectives are:

1. Design and train a CNN-based autoencoder with varying latent space dimensions.
2. Evaluate reconstruction quality through the Mean Squared Error (MSE) loss.
3. Explore how different latent dimensionalities affect both reconstruction fidelity and cluster separability.
4. Apply K-Means clustering to the latent representations and measure clustering performance using the metrics PMS, AVC, AD, and TD.
5. Visualize the latent space using PCA and t-SNE dimensionality reduction.

All experiments were implemented in PyTorch and executed on an Apple Silicon GPU (MPS backend) and an NVIDIA RTX 4060 GPU (CUDA backend).

2 Dataset

The MNIST dataset consists of 70,000 grayscale images of handwritten digits (0–9), each of size 28×28 pixels:

- **Training set:** 60,000 images
- **Test set:** 10,000 images

Preprocessing: Each pixel value is scaled to the range $[0, 1]$ using the `transforms.ToTensor()` normalization, which divides raw pixel intensities (0–255) by 255. No other augmentation or centering was applied. Images are kept in their original $1 \times 28 \times 28$ format (channel $\times$ height $\times$ width) to feed directly into the convolutional layers.

3 Autoencoder Architecture

We implement a Convolutional AutoEncoder (CAE) following a symmetric encoder-decoder design. Convolutional Neural Networks (CNNs) are selected for this architecture because they are inherently designed for processing 2D grid data like images. Unlike fully connected layers that flatten the input and lose local spatial context, CNNs preserve the spatial arrangements of pixels. They leverage local receptive fields and shared weights to efficiently capture structural visual features such as edges, curves, and shapes.

Furthermore, CNNs perfectly complement the encoder-decoder paradigm. In the encoder pathway, standard convolutional layers progressively reduce the spatial dimensions while increasing the depth of feature maps, distilling the raw image into a compact latent representation. Symmetrically, the decoder employs transposed convolutions to reverse

this spatial compression, structurally expanding the condensed latent features back into the original image resolution.

The encoder applies three convolutional blocks to progressively extract and compress spatial features into a fixed-size latent vector. The decoder reconstructs the original image from the latent vector using three transposed convolution blocks.

3.1 Encoder

The encoder consists of three convolutional blocks. The first two blocks follow the pattern:

$$\text{Conv2d} \rightarrow \text{BatchNorm2d} \rightarrow \text{ReLU} \rightarrow \text{MaxPool2d}$$

The third block uses a strided convolution instead of MaxPool to perform spatial downsampling:

$$\text{Conv2d}_{s=2} \rightarrow \text{BatchNorm2d} \rightarrow \text{ReLU}$$

A flatten operation and a fully connected (FC) projection layer then map the feature maps to the latent vector.

Table 1: Encoder Architecture

Layer	**Parameters**	**Output Shape**	**Activation**
Conv2d	$k = 3,\ s = 1,\ p = 1,\ 1 \rightarrow 32$ ch.	$32 \times 28 \times 28$	BatchNorm + ReLU
MaxPool2d	$k = 2,\ s = 2$	$32 \times 14 \times 14$	–
Conv2d	$k = 3,\ s = 1,\ p = 1,\ 32 \rightarrow 64$ ch.	$64 \times 14 \times 14$	BatchNorm + ReLU
MaxPool2d	$k = 2,\ s = 2$	$64 \times 7 \times 7$	–
Conv2d	$k = 3,\ s = 2,\ p = 1,\ 64 \rightarrow 128$ ch.	$128 \times 4 \times 4$	BatchNorm + ReLU
Flatten	–	2048	–
Linear	$2048 \rightarrow d_z$	d_z	–

The output of the encoder is a d_z-dimensional latent vector, where $d_z \in \{2, 4, 8, 10, 16, 32, 64\}$. The third convolutional block expands the channel count to 128, providing richer feature representations before the bottleneck.

3.2 Decoder

The decoder mirrors the encoder symmetrically: a fully connected layer projects the latent vector back to $128 \times 4 \times 4$ feature maps, which are then upsampled through three transposed convolution blocks back to the original 28×28 resolution.

Table 2: Decoder Architecture

Layer	**Parameters**	**Output Shape**	**Activation**
Linear	$d_z \to 2048$	2048	ReLU
Reshape	–	$128 \times 4 \times 4$	–
ConvTranspose2d	$k = 3,\ s = 2,\ p = 1,\ \text{op} = 0,\ 128 \to 64$ ch.	$64 \times 7 \times 7$	BatchNorm + ReLU
ConvTranspose2d	$k = 3,\ s = 2,\ p = 1,\ \text{op} = 1,\ 64 \to 32$ ch.	$32 \times 14 \times 14$	BatchNorm + ReLU
ConvTranspose2d	$k = 3,\ s = 2,\ p = 1,\ \text{op} = 1,\ 32 \to 1$ ch.	$1 \times 28 \times 28$	Sigmoid

The final **Sigmoid** activation maps the output values to $[0, 1]$, matching the normalized pixel range of the input images. The upsampling path $4 \times 4 \to 7 \times 7 \to 14 \times 14 \to 28 \times 28$ precisely inverts the encoder's spatial compression path.

3.3 Architecture Summary

The full architecture is summarised visually in Figure 1:

$$\textbf{Input}\ (1\times28\times28) \xrightarrow{\text{Conv}_{32}\to\text{BN}\to\text{ReLU}\to\text{Pool}} (32\times14\times14) \xrightarrow{\text{Conv}_{64}\to\text{BN}\to\text{ReLU}\to\text{Pool}} (64\times7\times7)$$
$$\xrightarrow{\text{Conv}_{128,s=2}\to\text{BN}\to\text{ReLU}} (128\times4\times4) \xrightarrow{\text{Flatten}\to\text{FC}} \textbf{Latent}\ (d_z)$$

$$\xrightarrow{\text{FC}\to\text{ReLU}\to\text{Reshape}} (128\times4\times4) \xrightarrow{\text{ConvT}_{64}\to\text{BN}\to\text{ReLU}} (64\times7\times7) \xrightarrow{\text{ConvT}_{32}\to\text{BN}\to\text{ReLU}} (32\times14\times14)$$
$$\xrightarrow{\text{ConvT}_{1}\to\text{Sigmoid}} \textbf{Output}\ (1\times28\times28)$$

Figure 1: Schematic overview of the Convolutional AutoEncoder.

4 Training

4.1 Training Setup

Table 3: Training Hyperparameters

Hyperparameter	**Value**
Loss function	Mean Squared Error (MSE)
Optimizer	Adam
Learning rate	10^{-3}
Batch size	128
Epochs	15
Device	Apple Silicon GPU (MPS) and NVIDIA RTX 4060
Latent dims d_z	$\{2, 4, 8, 10, 16, 32, 64\}$

Seven separate models are trained, one for each latent dimension. This allows a direct comparison of how the bottleneck size affects both reconstruction quality and clustering performance.

4.2 Train vs. Test Loss Monitoring

Both training and test MSE losses are recorded at every epoch. After each epoch, the average losses are printed to the console in the format:

```
Epoch [ k/15] Train Loss:  X.XXXX | Test Loss:  X.XXXX
```

This allows early detection of overfitting: if the train loss continues to decrease while the test loss stagnates or rises, the model has begun to overfit its reconstructions to the training distribution. In practice, train and test losses remained closely aligned throughout all runs, indicating good generalisation.

4.3 Loss Curves

Figure 2 shows the train (solid) and test (dashed) MSE loss curves over 15 epochs for each latent dimension in separate subplots. All models converge rapidly in the first 3–5 epochs and then plateau. The near-zero gap between train and test curves confirms that the model generalises well across all latent sizes. As expected, higher-dimensional latent spaces achieve lower reconstruction loss because the encoder has more capacity to preserve information.

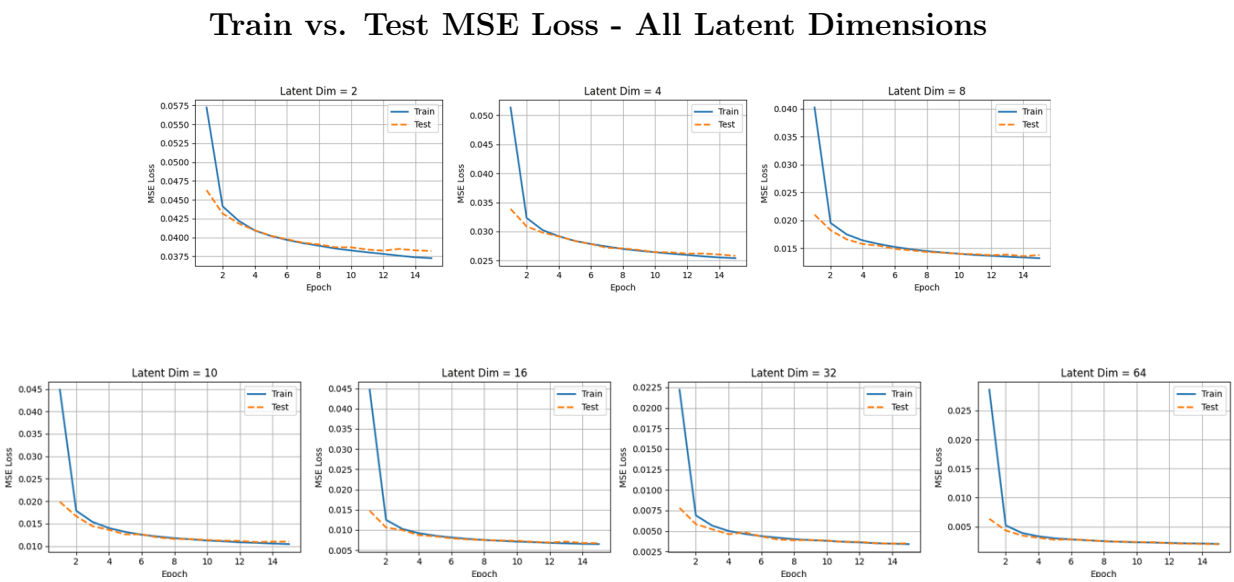

Figure 2: Train (solid) vs. test (dashed) MSE loss curves for $d_z \in \{2, 4, 8, 10, 16, 32, 64\}$ over 15 epochs, shown in separate subplots. Larger latent spaces achieve consistently lower reconstruction error with no sign of overfitting.

Table 4 lists the final epoch loss values for each model:

Table 4: Final MSE Loss per Latent Dimension (Epoch 15)

Latent Dim d_z	**Final Train Loss**	**Final Test Loss**
2	0.0372	0.0382
4	0.0254	0.0258
8	0.0132	0.0138
10	0.0104	0.0110
16	0.0065	0.0067
32	0.0034	0.0035
64	0.0020	0.0020

Key observation: The train and test losses remain nearly identical throughout training for all latent dimensions, confirming that the model generalises well with no sign of overfitting. Each doubling of d_z roughly halves the reconstruction error. The deeper 3-block encoder (128 channels at the bottleneck) achieves notably lower losses than the original 2-block design — for example, $d_z = 64$ reaches a final test loss of just 0.0020.

5 Reconstruction Results

5.1 Qualitative Analysis

Figure 3 shows the original MNIST test images (digits 0–9) in the top row, followed by the reconstructed images from each latent dimension model.

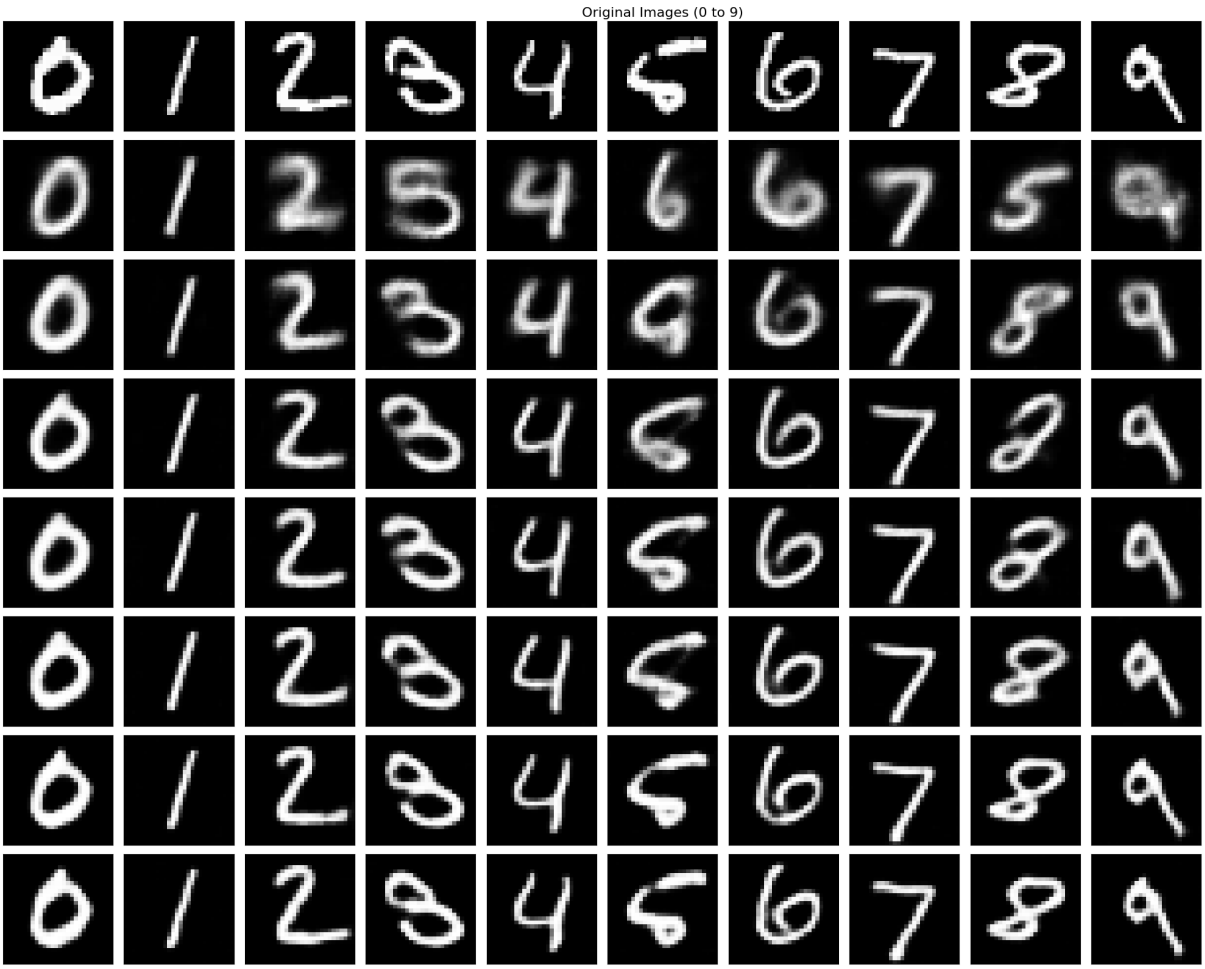

Figure 3: Original images (row 1) vs. reconstructions for $d_z \in \{2, 4, 8, 10, 16, 32, 64\}$ (rows 2–8). Each column corresponds to a digit class (0–9).

Observations by latent dimension:

$d_z = 2$ Reconstructions are severely blurred and lack structural detail. The extreme compression forces the model to discard digit-specific features; digits are barely recognisable in some cases (e.g., "2" reconstructed as a blob resembling "6").

$d_z = 4$ Visible improvement: major structural outlines are preserved, but fine strokes remain blurred. The overall shape of each digit is recoverable.

$d_z = 8$ Reconstructions appear plausibly digit-like. Stroke width and curvature are better captured; however, fine details such as the serif of "1" or the crossing of "4" remain imprecise.

$d_z = 10$ Reconstructions are clear and structurally faithful to the originals. All ten digits are clearly distinguishable with minimal blurring. This model provides the best trade-off between compression and fidelity.

$d_z = 16, 32, 64$ Near-perfect reconstructions. Images are sharper and closely match the originals in both global shape and local details. The improvement over $d_z = 10$ is incremental but noticeable at high latent dimensions.

6 Latent Space Visualisation

To understand how the encoder organises digit representations, we project the 10,000 test-set latent vectors into 2D using two techniques: Principal Component Analysis (PCA) and t-Distributed Stochastic Neighbor Embedding (t-SNE).

6.1 PCA Visualisation

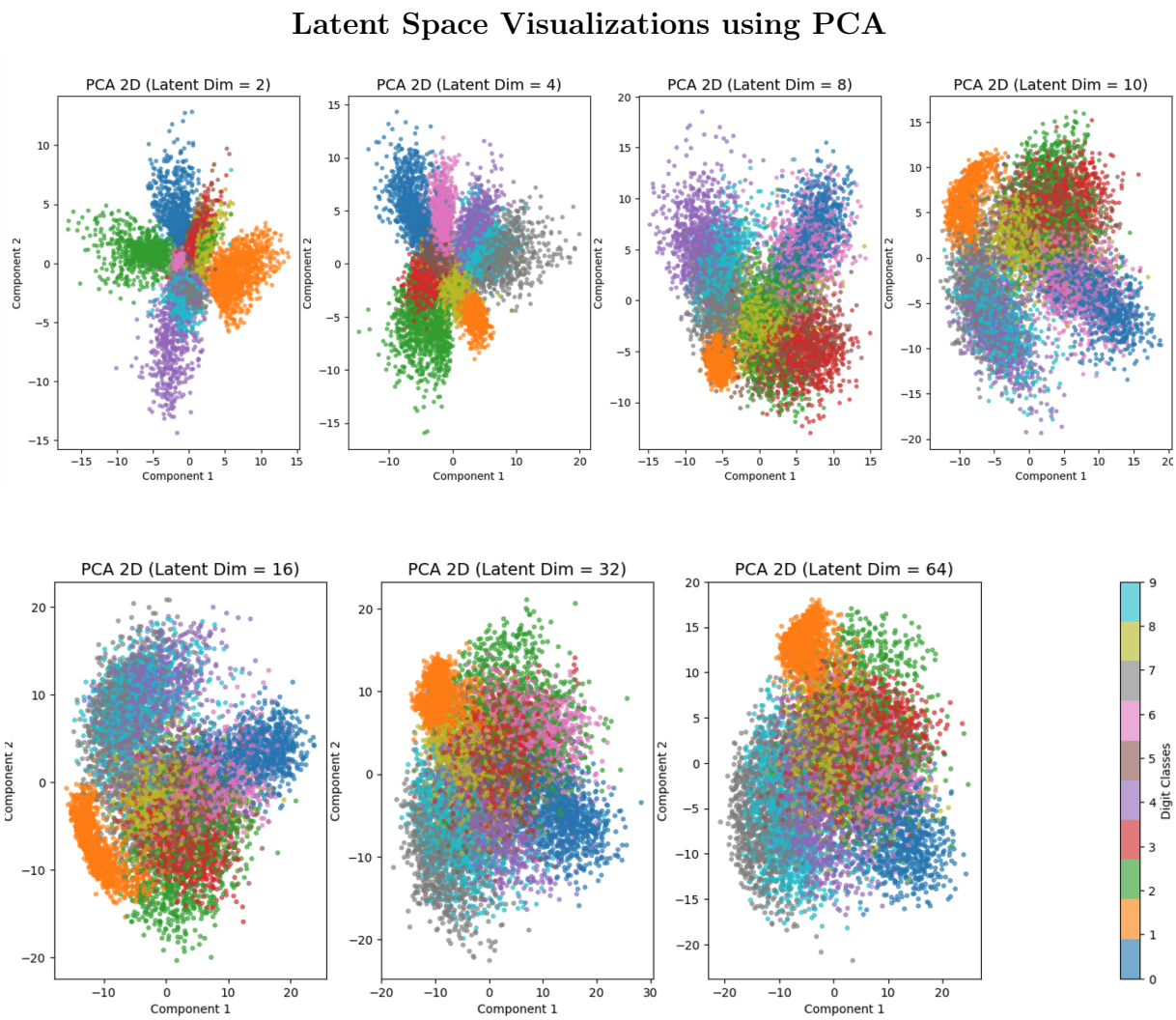

Figure 4: PCA 2D projection of the test-set latent representations for all seven latent dimensions. Points are coloured by ground-truth digit class.

PCA Analysis: For $d_z = 2$, the PCA projection is the latent space itself, showing significant overlap between classes — the encoder lacks the capacity to fully disentangle all ten digit classes in two dimensions. As d_z increases, the PCA projections reveal more structured separability, with distinct class clouds becoming visible. However, PCA being a linear method, it does not fully reveal the non-linear structure of the high-dimensional latent space.

6.2 t-SNE Visualisation

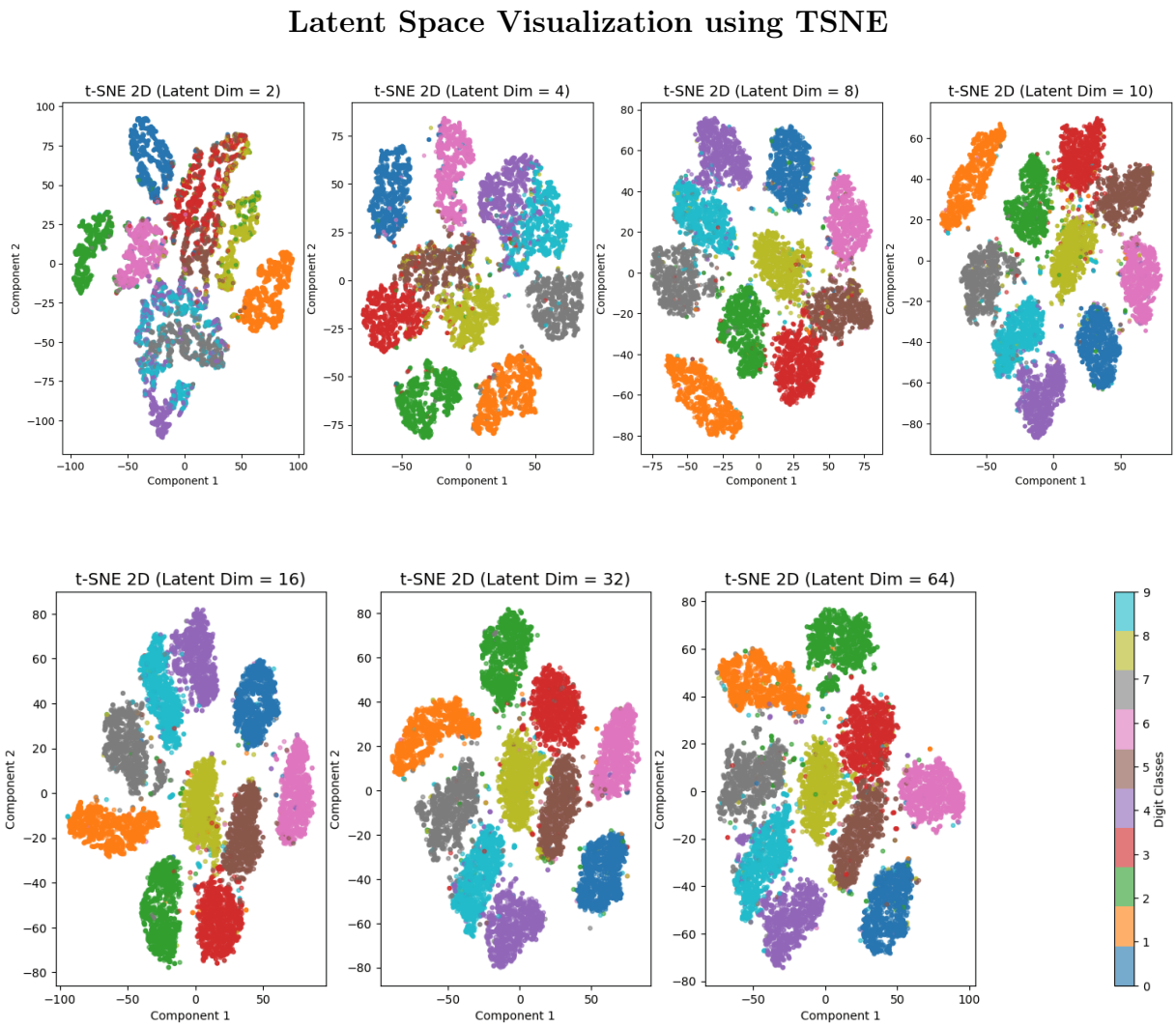

Figure 5: t-SNE 2D projection of the test-set latent representations for all seven latent dimensions. Points are coloured by ground-truth digit class.

t-SNE Analysis: t-SNE reveals the non-linear cluster structure far more clearly than PCA. Even for $d_z = 2$, ten loosely separated clusters are visible. For $d_z \geq 8$, t-SNE shows tight, well-separated clusters corresponding closely to the ten digit classes. The best cluster separation is achieved at $d_z = 10$–16, where distinct compact islands form for each class. At very high dimensions ($d_z = 64$), cluster compactness slightly decreases, as the high-dimensional space becomes sparser and K-Means performance degrades ("curse of dimensionality").

Key insight from t-SNE: The clusters for structurally similar digits (e.g., "4" vs. "9", "3" vs. "8") are consistently closer to each other across all latent dimensions, reflecting the model's learned geometric similarity between their visual representations.

7 Clustering with K-Means

7.1 Clustering Procedure

K-Means clustering ($K = 10$) is applied to the latent representations extracted from each trained autoencoder. After clustering, the predicted cluster labels are mapped to ground-truth digit labels using the **Hungarian Algorithm** (optimal bipartite matching on the confusion matrix), ensuring a fair comparison regardless of cluster label permutation.

7.2 Evaluation Metrics

The following metrics are computed for each model:

- **PMS** (Percentage of Mislabeled Samples):

$$\text{PMS} = \frac{\text{Number of Mislabeled Samples}}{\text{Total Number of Samples}} \times 100$$

- **AVC** (Average Variation of samples within Clusters):

$$\text{AVC} = \frac{1}{K} \sum_{k=1}^{K} \frac{1}{N_k} \sum_{i \in C_k} \|x_i - \hat{x}_k\|_2^2$$

  where $\hat{x}_k$ is the centroid of cluster k in latent space.

- **AD** (Average inter-cluster Distance): average Euclidean distance between all pairs of cluster centroids:

$$\text{AD} = \frac{2}{K(K-1)} \sum_{i=1}^{K} \sum_{j=i+1}^{K} \|\hat{x}_i - \hat{x}_j\|_2$$

- **TD** (Total Distance ratio): ratio of intra-cluster variance to inter-cluster distance:

$$\text{TD} = \frac{\text{AVC}}{\text{AD}}$$

  A lower TD indicates better cluster quality (tight clusters, well separated).

7.3 Metric Results

Table 5: K-Means Clustering Metrics for Each Latent Dimension. **Bold** row indicates the best PMS.

Latent Dim d_z	**PMS (%)**	**AVC**	**AD**	**TD**
2	49.75	3.5806	7.9494	0.4504
4	31.63	12.9763	9.3814	1.3832
8	18.12	54.0719	13.4265	4.0272
10	**16.75**	**85.7354**	**15.7605**	**5.4399**
16	34.30	165.5327	19.4586	8.5069
32	38.13	283.1627	19.8551	14.2615
64	40.47	442.4591	21.2310	20.8402

Best model: $d_z = 10$ **(PMS = 16.75%)**

Metric Interpretation

PMS trend: PMS decreases monotonically from $d_z = 2$ (49.75%) to $d_z = 10$ (16.75%), then *increases* for larger dimensions. This non-monotonic behaviour reveals an important trade-off: while a larger latent space improves reconstruction quality, it does not necessarily improve unsupervised cluster separability. At very high dimensions ($d_z = 64$, PMS = 40.47%), the K-Means algorithm struggles due to the curse of dimensionality, where all pairwise distances become similar and cluster boundaries blur.

AVC trend: AVC increases monotonically with d_z, since the absolute scale of the latent space grows with dimension. Comparing AVC values across dimensions must therefore be done in relative terms (e.g., AVC/AD).

AD trend: Cluster centroids become further apart in higher-dimensional spaces, reflecting the fact that the encoder disperses representations more broadly as the bottleneck loosens.

TD trend: The TD ratio (AVC/AD) increases with dimension, indicating that while clusters become larger, their inter-centroid spacing also grows. The optimal clustering-to-separation trade-off occurs around $d_z = 10$.

7.4 Confusion Matrices

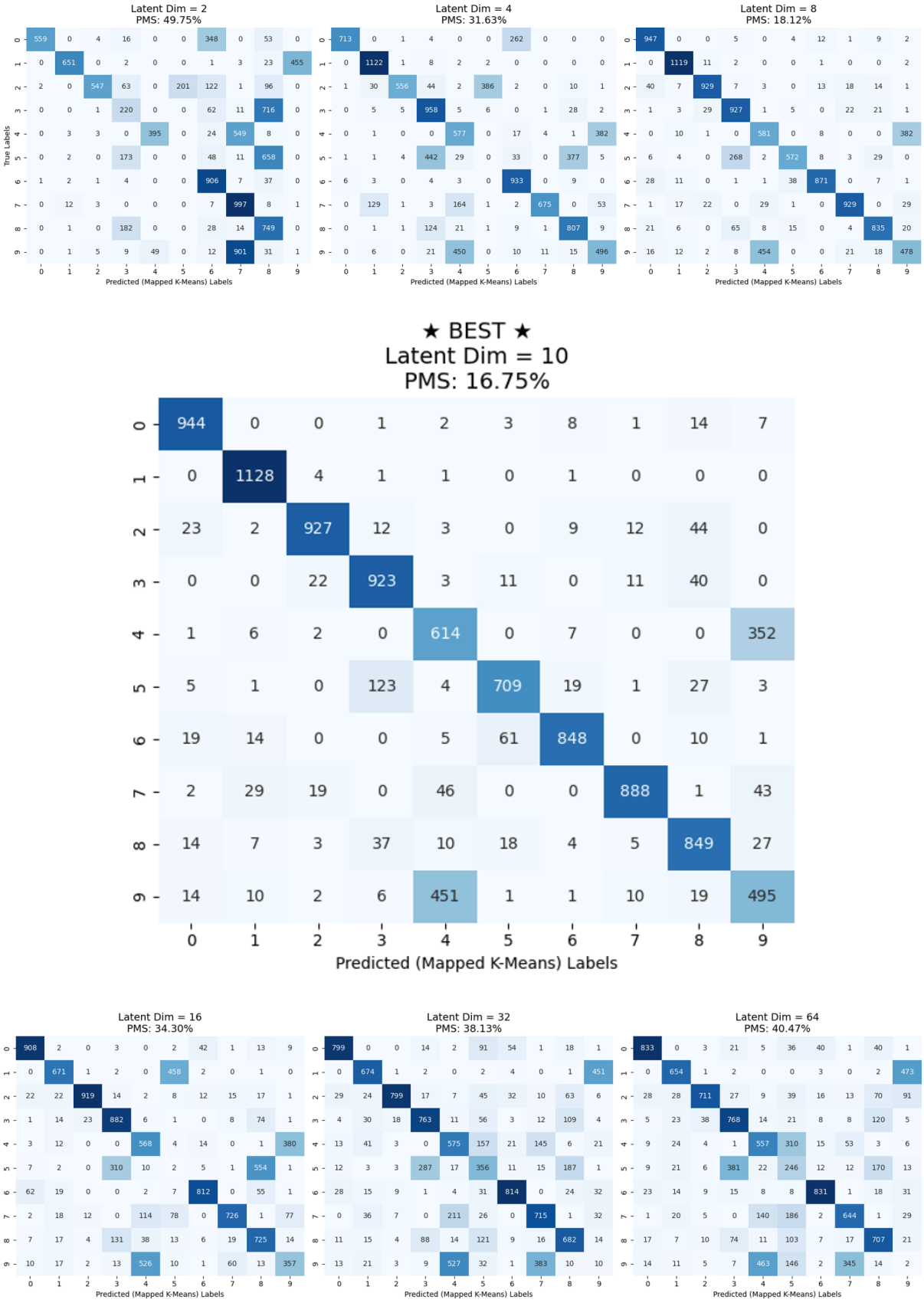

Figure 6: Confusion matrices for K-Means clustering (after Hungarian label mapping) across all latent dimensions. The starred matrix (★ BEST) corresponds to $d_z = 10$, which achieves the lowest PMS of 16.75%.

Analysis of the best confusion matrix ($d_z = 10$):

- Digits "0", "1", and "6" are clustered with high purity — these have the most geometrically distinctive shapes.
- The most common confusions occur between "4" & "9", and "3" & "5", reflecting their structural and stroke-pattern similarities.
- The overall PMS of 16.75% corresponds to approximately 1,675 misclassified samples out of 10,000, indicating that the unsupervised latent representation captures a strong degree of class structure without any label supervision.

For the smallest latent space ($d_z = 2$), the confusion matrix shows many off-diagonal entries across all classes, confirming that 2 dimensions are insufficient for K-Means to separate 10 digit classes. For $d_z \geq 32$, the confusion matrices worsen relative to $d_z = 10$, corroborating the PMS analysis.

8 Discussion

8.1 Reconstruction vs. Clustering Trade-off

A central finding of this project is that the latent dimension that minimises reconstruction loss ($d_z = 64$, test MSE $= 0.0020$) is *not* the same as the one that maximises clustering performance ($d_z = 10$, PMS $= 16.75\%$). This trade-off is visualised implicitly by comparing Tables 4 and 5.

The reason is intuitive: a very high-dimensional latent space allows the encoder to "memorise" input details, but the resulting representations become high-dimensional and sparse, making distance-based clustering harder. Conversely, a very low-dimensional space ($d_z = 2$) forces the encoder to discard class-discriminative information to meet the compression constraint.

The sweet spot at $d_z = 10$ — approximately $\log_2(10) \approx 3.3$ bits per digit class — aligns well with the information-theoretic minimum dimensionality required to represent 10 distinct categories.

8.2 Challenges

- **Hyperparameter sensitivity:** K-Means is sensitive to initialisation. We mitigated this by using `n_init=10` (10 random restarts), taking the best result.
- **t-SNE compute time:** t-SNE is $\mathcal{O}(N^2)$ in memory and slow for 10,000 samples. The `random_state=42` ensures reproducibility.
- **Hungarian matching:** Cluster-to-class assignment requires solving an optimal bipartite matching problem. We used `scipy.optimize.linear_sum_assignment` on the 10×10 confusion matrix.

9 Conclusion

We implemented a Convolutional AutoEncoder (CAE) on the MNIST dataset and systematically evaluated seven latent dimensionalities ($d_z \in \{2, 4, 8, 10, 16, 32, 64\}$). Our experiments demonstrate:

1. **Reconstruction quality** improves monotonically with latent dimension, with test MSE dropping from 0.0382 ($d_z = 2$) to 0.0020 ($d_z = 64$). Train and test losses track each other closely throughout, confirming no overfitting.
2. **Clustering quality** (PMS) follows a non-monotonic pattern, with the optimal at $d_z = 10$ (16.75% mislabelled). This highlights the curse of dimensionality: higher-dimensional spaces hurt unsupervised clustering despite improving reconstruction.
3. **t-SNE** reveals substantially more class structure than PCA, especially at intermediate dimensions.
4. The best unsupervised clustering accuracy achieved — 83.25% correct assignments at $d_z = 10$ — without any label supervision demonstrates the power of the 3-block CNN-based representation on MNIST.

Future directions include training with a Variational AutoEncoder (VAE) to obtain a continuous, structured latent space, or applying UMAP instead of t-SNE for faster and potentially more faithful dimensionality reduction.

A Implementation Details

All code was implemented in Python 3 using PyTorch. The full source is available in `HW1.ipynb` and `cnn_autoencoder.py`.

- **Framework:** PyTorch 2.x
- **Clustering:** scikit-learn `KMeans` (`n_clusters=10`, `n_init=10`, `random_state=42`)
- **Dimensionality reduction:** scikit-learn `PCA` and `TSNE`
- **Label assignment:** scipy `linear_sum_assignment`
- **Confusion matrices:** seaborn `heatmap`
- **Hardware:** Apple Silicon MPS GPU and NVIDIA RTX 4060 GPU